



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE MM.FF.NN.

CORSO DI LAUREA IN INFORMATICA

Tesi di laurea triennale

Progettazione e sviluppo di un framework per la verifica
della conformità e della sicurezza dei protocolli di telefonia
4G/5G su dispositivi COTS

2024-2025

RELATORE:

Pierpaolo Loreti

CANDIDATO:

Marco Altomare

CORRELATORE:

Francesco Mancini

Indice

Introduzione	1
1 Background e contesto	2
1.1 Il contesto 4G e 5G	2
1.2 Il problema del testing su dispositivi COTS	3
1.3 Conformità, robustezza e sicurezza	4
1.4 Obiettivi e contributi della tesi	5
1.5 Concetti tecnici di base	6
1.6 Architettura IMS	6
1.7 Il protocollo SIP	7
1.8 Flussi rilevanti per il framework	7
1.9 Sintesi	8
2 Requisiti e progettazione	9
2.1 Contesto del Test program	9
2.2 Requisiti funzionali	10
2.3 Requisiti non funzionali	12
2.4 Casi d'uso	12
2.5 Modalità di esecuzione	13
2.6 Scelte progettuali	14

2.7	Plugin <code>sip-proxy-http</code>	15
2.8	Plugin <code>kamailio-forward</code>	16
2.9	Sintesi	17
3	Architettura del framework	18
3.1	Componenti principali del framework	18
3.2	Architettura del Test program	18
3.3	Sistema a plugin per i backend di trasporto	18
3.4	Backend <code>sip-proxy-http</code> e tester API	18
3.5	Discovery dei dispositivi	18
3.6	Backend <code>kamailio-forward</code>	18
3.7	Flussi di esecuzione	18
3.8	Deployment nel laboratorio Docker	18
3.9	Sintesi	18
4	Implementazione	19
4.1	Scelte di implementazione e motivazioni	20
4.2	Stack tecnologico e organizzazione del codice	22
4.3	Configurazione e modello dei dati	22
4.4	CLI e flussi operativi	23
4.5	Motore di esecuzione dei test	24
4.6	Gestione dei messaggi SIP	25
4.7	Backend di trasporto e protocolli di comunicazione	26
4.8	Piattaforma Docker e deployment	27
4.9	Validazione automatica e manutenzione	28
4.10	Sintesi	28

5	Validazione e risultati	29
6	Conclusioni e sviluppi futuri	30
	Elenco delle figure	31
	Elenco dei codici	31
	Elenco degli algoritmi	31

Ringraziamenti

Ringraziamenti a relatori, correlatori, colleghi e persone che hanno supportato il lavoro.

Abstract

Questa tesi presenta un framework software per la verifica della conformità e della sicurezza dei protocolli di telefonia 4G/5G su dispositivi COTS. Il lavoro integra un motore di test configurabile e un componente di proxy per l'intercettazione e la manipolazione controllata del traffico, con l'obiettivo di supportare analisi automatizzate e ripetibili su stack IMS proprietari.

Introduzione

La crescente diffusione dei servizi vocali su reti 4G e 5G rende necessario disporre di strumenti software in grado di verificare il comportamento degli stack IMS presenti nei dispositivi commerciali. Questa tesi affronta il problema proponendo un framework modulare per l'esecuzione di test di conformità e sicurezza, con attenzione alla flessibilità, alla manutenibilità e alla possibilità di integrare nuovi scenari di verifica.

Capitolo 1

Background e contesto

In questo capitolo viene inquadrato il problema del testing di stack IP Multimedia Subsystem (IMS) su dispositivi Commercial Off-The-Shelf (COTS) e vengono richiamati solo i concetti indispensabili, in particolare il protocollo Session Initiation Protocol (SIP), per comprendere il framework proposto.

1.1 Il contesto 4G e 5G

Nelle reti di quarta generazione (4G), basate su *Long Term Evolution* (LTE), e di quinta generazione (5G), basate su *New Radio* (NR), la telefonia non è più un servizio dedicato del dominio circuito, ma viene erogata come servizio su *Internet Protocol* (IP) tramite *Voice over LTE* (VoLTE) o *Voice over NR* (VoNR). In entrambi i casi la segnalazione voce passa tipicamente attraverso un'infrastruttura IMS, in cui i messaggi SIP definiscono registrazione, instaurazione della sessione e relative politiche di instradamento.

Per il lavoro svolto in questa tesi il dettaglio dell'architettura del core (*Evolved Packet Core*, EPC, nel 4G; *5G Core*, 5GC, nel 5G) è rilevante solo nella misura in cui condiziona il percorso della segnalazione. Il punto centrale è che gran parte del comportamento osservabile di un terminale COTS, e quindi verificabile dall'esterno,

emerge a livello IMS/SIP: è su questo piano che si concentrano i test di conformità e robustezza.

In particolare, anche se i meccanismi di mobilità, autenticazione e policy cambiano tra 4G e 5G, la catena di segnalazione IMS rimane il punto in cui il terminale espone il proprio comportamento applicativo. Per questa ragione il framework assume la segnalazione SIP come superficie osservabile e manipolabile: variare in modo controllato uno scambio SIP consente di esercitare il parsing e le logiche interne del dispositivo senza richiedere accesso al suo ambiente di esecuzione.

1.2 Il problema del testing su dispositivi COTS

I dispositivi COTS (*Commercial Off-The-Shelf*) implementano stack IMS spesso proprietari e poco osservabili. In un contesto sperimentale questo crea un paradosso: i casi più interessanti da verificare sono proprio quelli in cui la segnalazione devia dal percorso nominale, ma il terminale resta una “scatola nera” e non può essere strumentato internamente.

Il testing tradizionale, basato su osservazione passiva o su strumenti pensati per stack modificabili, incontra tre limiti ricorrenti:

- **Osservabilità:** il comportamento interno del device deve essere inferito dai messaggi di rete.
- **Ripetibilità:** scenari complessi devono poter essere riprodotti a parità di condizioni, in modo documentabile.
- **Controllo:** per testare robustezza e sicurezza serve introdurre variazioni mirate senza rompere la coerenza del dialogo.

Ne consegue la necessità di un approccio “dall'esterno”: un'infrastruttura che consenta di orchestrare scenari e manipolare la segnalazione in modo controllato, senza richiedere modifiche al firmware del terminale o all'ambiente dell'operatore.

1.3 Conformità, robustezza e sicurezza

Nel dominio IMS, la conformità è legata al rispetto delle specifiche normative: l'architettura e le procedure di rete mobile sono descritte prevalentemente dal *3rd Generation Partnership Project* (3GPP), mentre la segnalazione SIP è standardizzata dall'*Internet Engineering Task Force* (IETF) tramite documenti *Request for Comments* (RFC). In pratica, un test di conformità verifica che, a fronte di uno stimolo noto (ad esempio un REGISTER o un INVITE), il dispositivo produca messaggi corretti e coerenti con lo stato del dialogo.

La robustezza riguarda invece la capacità del terminale di gestire input inattesi senza comportamenti anomali. In un protocollo testuale come SIP, piccole variazioni nella forma dei messaggi (intestazioni duplicate, valori incoerenti, campi opzionali mancanti) possono attivare percorsi di parsing meno battuti, con effetti osservabili anche senza accesso al codice sorgente. La verifica della robustezza si basa quindi su modifiche mirate e ripetibili del traffico, seguite dall'osservazione di risposta, timeout o comportamenti divergenti.

La sicurezza si intreccia naturalmente con la robustezza: molte vulnerabilità in stack di segnalazione derivano da assunzioni implicite sul formato dei messaggi e sulla sequenza degli scambi. Un framework di test che preserva il messaggio raw e consente di applicare trasformazioni controllate costituisce quindi una base utile sia per verifiche di conformità sia per campagne di analisi più orientate alla ricerca di condizioni limite.

1.4 Obiettivi e contributi della tesi

L'obiettivo di questa tesi è progettare e realizzare un framework che renda il testing di stack IMS su dispositivi COTS *pratico e riusabile*: non un insieme di prove ad hoc, ma una base software capace di supportare campagne di verifica ripetibili e confrontabili.

I contributi principali del lavoro sono concreti e verificabili:

- sviluppo del *Test program*, una *Command Line Interface* (CLI) in Python per validare suite di test, eseguire verifiche di conformità su un singolo device e confrontare terminali in modalità differenziale;
- definizione di un formato dichiarativo in *YAML Ain't Markup Language* (YAML) per descrivere messaggi SIP raw, modifiche controllate, attese e parametri di runtime;
- progettazione di un sistema a plugin per i backend di trasporto (es. 'sip-proxy-http', 'kamailio-forward', backend simulato), così da separare logica di test e meccanismo di comunicazione;
- integrazione con un laboratorio replicabile basato su Docker e su componenti 4G/5G+IMS, includendo discovery dei dispositivi e gestione di regole di *live edit* sul traffico;
- generazione di report sia testuali sia *JavaScript Object Notation* (JSON) per rendere i risultati archiviabili e confrontabili.

In sintesi, il valore del framework è trasformare un'attività tipicamente manuale e fragile (testare un device COTS in IMS) in un processo più controllato, ripetibile e misurabile.

1.5 Concetti tecnici di base

Le sezioni seguenti richiamano i concetti minimi necessari a leggere la tesi: che cos'è IMS, quali elementi della catena sono rilevanti e quali proprietà di SIP rendono possibile (ma delicata) l'ispezione e la manipolazione dei messaggi durante i test.

1.6 Architettura IMS

L'*IP Multimedia Subsystem* (IMS) è l'architettura usata per erogare servizi multimediali, inclusa la voce, sopra un'infrastruttura IP. Il suo ruolo è gestire registrazione, autenticazione e controllo di sessione, separando la logica di servizio dal livello di accesso radio.

Dal punto di vista operativo, il terminale interagisce con un *Proxy-Call Session Control Function* (P-CSCF) come primo punto di ingresso nel dominio IMS, e le richieste vengono poi gestite da nodi come *Interrogating-Call Session Control Function* (I-CSCF) e *Serving-Call Session Control Function* (S-CSCF). Le funzioni di identità e profilo (ad esempio *Home Subscriber Server* (HSS), o componenti equivalenti nel dominio 5G) supportano autenticazione e autorizzazioni.

Per questa tesi è fondamentale la separazione tra piani: la segnalazione (SIP) controlla registrazione e sessioni, mentre i flussi media vengono negoziati separatamente (tipicamente via *Session Description Protocol* (SDP)) e trasportati fuori banda. Il framework interviene sul piano di segnalazione, mantenendo per quanto possibile invariato il trasporto media.

1.7 Il protocollo SIP

SIP (*Session Initiation Protocol*) è il protocollo testuale usato da IMS per stabilire, modificare e terminare sessioni. I metodi più rilevanti nel contesto della tesi sono REGISTER (registrazione), INVITE/ACK (instaurazione della chiamata) e BYE (terminazione), oltre a CANCEL e OPTIONS per scenari specifici.

La natura testuale di SIP facilita l'ispezione del traffico, ma impone anche attenzione: intestazioni come Via, Route/Record-Route, Contact, Call-ID (*Call Identifier*), CSeq (*Command Sequence*) e i campi From/To determinano routing e stato del dialogo. Alterare un singolo campo può rendere il messaggio non più correlabile, o cambiare il percorso nella catena IMS. Per questo il framework adotta un approccio conservativo: tratta il messaggio come testo raw e applica trasformazioni solo quando esplicitamente richiesto dalla suite.

1.8 Flussi rilevanti per il framework

Dal punto di vista dei test, i flussi più importanti sono tre. Il primo è la registrazione SIP, che rende il dispositivo raggiungibile. Il secondo è la segnalazione di sessione, cioè l'instaurazione e la chiusura della chiamata con i messaggi correlati. Il terzo è la negoziazione dei parametri media tramite SDP, che deve rimanere coerente con il comportamento osservato sul piano di segnalazione.

Operativamente, il proxy di test si colloca sul percorso della segnalazione per osservare e applicare regole di manipolazione controllata, mentre il motore di test orchestra l'ordine dei casi, gestisce timeout e attese e produce report. In questo modo diventano eseguibili sia test di conformità sia test di robustezza e sicurezza senza intervenire direttamente sul terminale.

1.9 Sintesi

Il contesto 4G/5G motiva la centralità della segnalazione IMS/SIP per i servizi vocali su IP; la natura “black-box” dei dispositivi COTS rende però difficile un testing rigoroso senza un’infrastruttura dedicata. Il framework sviluppato in questa tesi fornisce tale infrastruttura, combinando orchestrazione dei test e controllo del traffico.

Queste nozioni costituiscono la base per i capitoli successivi: nel Capitolo 2 verranno tradotte in requisiti, casi d’uso e scelte progettuali del framework; nei capitoli seguenti saranno poi riprese per descrivere architettura, implementazione e validazione.

Capitolo 2

Requisiti e progettazione

Questo capitolo formalizza i requisiti del framework e descrive le scelte progettuali che hanno guidato lo sviluppo del Test program e dei backend di trasporto. L'obiettivo è rendere esplicito il legame tra vincoli del dominio IMS/SIP, casistica di test considerata e componenti software realizzati.

2.1 Contesto del Test program

Il Test program è stato progettato per rendere il testing di stack IMS su dispositivi COTS un processo ripetibile e documentabile. Il punto di partenza è la natura “black-box” dei terminali: non è possibile strumentarli internamente e, in molti casi, non è possibile modificare la rete dell'operatore o i nodi IMS reali coinvolti nel laboratorio. Di conseguenza, la verifica deve essere condotta dall'esterno, osservando e generando traffico SIP in modo controllato.

Nel lavoro svolto, questo obiettivo si è tradotto nella realizzazione di tre elementi complementari:

- un linguaggio dichiarativo in YAML per descrivere casi di test basati su messaggi SIP raw, modifiche esplicite e condizioni di successo/fallimento;
- una CLI in Python (il Test program) che valida le suite, orchestra l'esecuzione,

gestisce timeout/attese e produce report;

- un insieme di backend di trasporto che collegano il motore di test al laboratorio (proxy con tester API, forwarding verso P-CSCF, backend simulato).

L'idea progettuale è separare tre responsabilità, così da mantenere riusabili i test anche quando cambia l'ambiente:

1. **Definizione dello scenario** (cosa inviare e cosa aspettarsi): descritta nella suite YAML.
2. **Trasporto ed esecuzione** (come il messaggio raggiunge il target e come si leggono le risposte): incapsulata nei plugin di trasporto.
3. **Valutazione e presentazione** (quali criteri decretano il successo e come si riportano i risultati): gestita dal motore e dai renderer di report.

Inoltre, oltre ai casi nominali (es. **REGISTER** e chiamate **INVITE/BYE**), la tesi considera scenari non nominali utili per la robustezza: header inconsistenti, valori limite, messaggi incompleti o varianti di formattazione. Per supportare questa casistica senza introdurre “correzioni” automatiche, il framework tratta il messaggio SIP come testo raw e applica trasformazioni solo quando esplicitamente richiesto dalla suite.

2.2 Requisiti funzionali

I requisiti funzionali sono stati derivati dai flussi IMS/SIP da verificare e dai vincoli operativi del laboratorio. Ogni requisito è stato tradotto in una funzionalità implementata e verificabile nel Test program.

- RF1 – Validazione delle suite** Il sistema deve validare le suite YAML prima dell'esecuzione (schema, campi obbligatori, duplicati, vincoli tra campi), in modo da ridurre fallimenti “a runtime” e rendere i risultati confrontabili.
- RF2 – Esecuzione di test di conformità** Deve essere possibile eseguire una suite contro un singolo dispositivo e verificare, per ogni caso, che la risposta osservata soddisfi le aspettative (codice, intestazioni, body e timing).
- RF3 – Confronto differenziale tra dispositivi** Deve essere possibile eseguire gli stessi casi su due o più terminali e produrre un confronto strutturato delle differenze, riducendo i falsi positivi dovuti a mere varianti di formattazione.
- RF4 – Gestione dell'inventario e discovery** Il framework deve supportare sia dispositivi configurati staticamente sia dispositivi scoperti dinamicamente durante l'attività di laboratorio (ad esempio osservando REGISTER), unificando il tutto in un inventario unico selezionabile da CLI.
- RF5 – Intercettazione e live editing** Deve essere possibile osservare e, quando richiesto, modificare selettivamente il traffico SIP “in transit” tramite regole controllate, così da implementare casi di robustezza e di sicurezza senza alterare il terminale.
- RF6 – Reporting leggibile e archiviabile** L'esecuzione deve produrre report testuali per l'operatore e report JSON per analisi successive, includendo esito per caso, motivazioni dei fallimenti e metadati utili alla tracciabilità.

2.3 Requisiti non funzionali

Oltre alla correttezza funzionale, il framework deve poter essere usato in campagne di test ripetute, evolvere nel tempo e restare debuggabile.

RNF1 – Ripetibilità A parità di suite e runtime, il comportamento del runner deve essere deterministico: timeout e attese sono configurabili, e l'ordine dei passi è guidato dalla suite.

RNF2 – Estendibilità L'aggiunta di un nuovo backend di trasporto non deve richiedere modifiche al motore di esecuzione, ma solo l'implementazione dell'interfaccia prevista e la registrazione del plugin.

RNF3 – Manutenibilità Modello dati, parsing/validazione, trasporto e logica di test devono restare separati per ridurre l'accoppiamento e rendere più semplice la manutenzione.

RNF4 – Portabilità Il sistema deve funzionare sia in sviluppo locale sia nel laboratorio Docker, mantenendo la configurazione esterna al codice e descritta in YAML.

RNF5 – Osservabilità In caso di fallimento, il sistema deve fornire abbastanza informazioni per diagnosticare il problema (messaggi inviati/osservati, motivazione della mismatch, contesto del device e del trasporto).

2.4 Casi d'uso

I casi d'uso seguenti descrivono come i requisiti vengono esercitati dall'operatore durante una sessione di test.

UC1 – Validare una suite

L'operatore esegue la modalità `validate` su un file YAML di test. Il risultato è un esito immediato (successo/errore) con indicazione precisa della regola violata.

UC2 – Eseguire una suite su un dispositivo

L'operatore seleziona un device (esplicitamente o via selezione interattiva) e avvia `run-standard`. Per ogni caso la CLI invia il messaggio (eventualmente dopo applicazione di modifiche) e valuta la risposta secondo le aspettative, producendo un report di conformità.

UC3 – Confrontare due o più dispositivi

L'operatore avvia `compare` (o seleziona più device in `run-standard`) e ottiene un report che evidenzia differenze nei codici di risposta, nelle intestazioni e nel body.

UC4 – Unire dispositivi statici e scoperti

L'operatore usa `list-devices` per visualizzare l'inventario complessivo: profili definiti nel runtime e profili scoperti dinamicamente (es. da `REGISTER` o tramite Open5GS), utilizzabili poi come target di esecuzione.

UC5 – Attivare regole di live edit

Durante una sessione, il tester abilita regole di intercettazione/modifica sul proxy (automaticamente attraverso la suite o manualmente via comandi `live-edit-list/live-edit-clear`) per riprodurre specifiche condizioni di errore o varianti di segnalazione.

2.5 Modalità di esecuzione

Il framework prevede più modalità operative, tutte esposte dalla CLI.

La modalità **validate** controlla la suite di test senza contattare alcun dispositivo. È il primo passo consigliato, perchè consente di individuare errori di struttura o vincoli mancanti prima di coinvolgere il laboratorio.

La modalità **run-standard** esegue i casi di una suite su un dispositivo. Se vengono selezionati più device, la CLI passa automaticamente a un confronto differenziale per evitare ambiguità sull'obiettivo dell'esecuzione.

La modalità **compare** esegue esplicitamente il confronto tra due o più dispositivi. Questa scelta rende chiara la semantica del comando e permette di trattare confronti su più terminali senza duplicare la logica del motore.

La modalità **list-devices** unisce dispositivi configurati e dispositivi scoperti, rendendo visibile all'operatore quale inventario sia effettivamente disponibile nel momento dell'esecuzione.

Le modalità **live-edit-list** e **live-edit-clear** servono a gestire lo stato del proxy e a mantenere pulito l'ambiente di test tra una sessione e l'altra.

2.6 Scelte progettuali

Le principali scelte progettuali riflettono i requisiti descritti sopra e sono state validate durante lo sviluppo del laboratorio.

La prima scelta è stata usare YAML come formato di configurazione delle suite e del runtime. Questo formato è leggibile da un operatore umano, ma abbastanza strutturato da poter essere validato con regole precise; inoltre permette di descrivere messaggi, modifiche, attese e parametri di esecuzione senza introdurre strumenti esterni.

La seconda scelta è stata introdurre un modello a plugin per i backend di trasporto. Il motore di test non deve conoscere i dettagli del backend **sip-proxy-http**, del

backend `stub` o dell'inoltro verso Kamailio: deve soltanto parlare con un'interfaccia comune che espone apertura, chiusura, invio e lettura delle risposte.

La terza scelta è stata mantenere il messaggio SIP come testo raw il più possibile. Questo approccio evita normalizzazioni involontarie e rende eseguibili anche scenari limite (robustezza/sicurezza) in cui la forma del messaggio conta quanto il suo contenuto semantico.

La quarta scelta è stata rendere la selezione dei dispositivi flessibile e indipendente dall'inventario statico. L'operatore può selezionare per id, nome, IP, IMSI o numero telefonico; quando il contesto lo consente, può inoltre basarsi su discovery dinamica per evitare configurazioni manuali fragili.

La quinta scelta è stata includere una comparazione robusta delle risposte, con normalizzazione di header e body. In questo modo il framework segnala differenze reali e limita i falsi positivi dovuti a varianti di formattazione non significative.

2.7 Plugin sip-proxy-http

Il plugin `sip-proxy-http` è il backend più vicino al proxy reale usato nel laboratorio. Il suo compito è inoltrare comandi HTTP al proxy, che a sua volta si occupa del traffico SIP verso i dispositivi e della raccolta delle risposte.

Questo backend espone un insieme di operazioni essenziali: health check, invio messaggi, lettura risposte, lettura dei dispositivi scoperti e gestione delle regole di live edit. Dal punto di vista del Test program, il trasporto è sincrono: il runner invia un JSON contenente il messaggio grezzo e un identificatore di correlazione, quindi attende la risposta con un timeout configurabile.

La discovery da **REGISTER** è particolarmente importante. In un laboratorio realistico i dispositivi possono apparire o scomparire nel tempo, e l'inventario non può

essere considerato statico. Per questo il backend integra i device scoperti nel medesimo elenco usato per la selezione interattiva o per la risoluzione dei selettori passati da CLI. Quando richiesto dall'esperimento, la discovery può anche essere ottenuta interrogando Open5GS, mantenendo invariata l'interfaccia vista dal runner.

Un'altra responsabilità del plugin è mantenere la coerenza tra il messaggio da testare e il target verso cui il proxy deve inoltrarlo. Quando il runtime lo richiede, la destinazione può essere derivata dal device stesso o da metadati specifici, senza costringere la suite a conoscere la topologia di rete sottostante.

2.8 Plugin kamailio-forward

Il plugin `kamailio-forward` rappresenta una modalità alternativa pensata per scenari in cui il Test program deve parlare con il P-CSCF e chiedergli di inoltrare il messaggio verso il terminale di destinazione. In questo caso il backend non usa il proxy come endpoint HTTP, ma come punto di instradamento SIP attraverso un'intestazione dedicata.

L'idea progettuale è che il Test program inserisca nel messaggio un header che identifica il target, mentre il P-CSCF riconosce quell'header e reinstrada il traffico verso il device indicato. Il backend si occupa inoltre di inserire (quando abilitato) un `Via` locale per correlare le risposte e di bufferizzare i messaggi ricevuti in attesa della lettura da parte del runner.

Questa soluzione è utile quando si vuole testare il comportamento della catena IMS mantenendo il P-CSCF come nodo effettivo di inoltro, ma introducendo allo stesso tempo un punto di iniezione controllato per i messaggi di test.

2.9 Sintesi

I requisiti e le scelte progettuali presentati in questo capitolo traducono un'esigenza centrale della tesi: rendere il testing IMS/SIP su dispositivi COTS ripetibile, estendibile e leggibile. La combinazione di suite dichiarative, motore di esecuzione e backend pluggabili consente di adattare il framework a più laboratori mantenendo invariata la descrizione dei casi di test.

I capitoli successivi descrivono come queste scelte si concretizzino nell'architettura del sistema, nell'implementazione dei moduli principali e nella validazione sperimentale.

Capitolo 3

Architettura del framework

Questo capitolo illustra l'architettura complessiva della soluzione, i componenti principali e le loro interazioni.

- 3.1 Componenti principali del framework
- 3.2 Architettura del Test program
- 3.3 Sistema a plugin per i backend di trasporto
- 3.4 Backend sip-proxy-http e tester API
- 3.5 Discovery dei dispositivi
- 3.6 Backend kamailio-forward
- 3.7 Flussi di esecuzione
- 3.8 Deployment nel laboratorio Docker
- 3.9 Sintesi

Capitolo 4

Implementazione

Questo capitolo presenta le principali scelte di implementazione, i moduli sviluppati e le soluzioni adottate per mantenere il codice estendibile e manutenibile.

L'architettura presentata nel capitolo precedente è stata tradotta in una soluzione concreta che integra programmazione Python, configurazioni dichiarative, script di inizializzazione e orchestrazione tramite Docker. L'obiettivo principale è stato costruire un sistema di test riusabile, capace di eseguire verifiche di conformità SIP su dispositivi IMS reali o simulati, ma anche di confrontare il comportamento di due terminali in modalità differenziale.

Una parte rilevante del lavoro è consistita nel trasformare scelte architetturali in decisioni operative verificabili. In questa prospettiva, l'implementazione non si limita a descrivere i componenti realizzati, ma mostra anche perché alcune soluzioni sono state preferite ad altre: l'uso di Python come linguaggio applicativo, l'adozione di YAML per la configurazione, l'integrazione del proxy nella rete Docker del laboratorio e la separazione tra logica di test e trasporto.

Dal punto di vista tecnologico il progetto si basa su Python 3.10 o superiore, sul caricamento di file YAML per la configurazione e su una suite di test automatici per la verifica del comportamento. La piattaforma di esecuzione è Docker, mentre i servizi

di rete e i componenti IMS sono descritti in modo dichiarativo e inizializzati con script che sostituiscono parametri e indirizzi nei template.

La scelta di separare la logica di test dal trasporto ha permesso di mantenere la soluzione indipendente dal contenuto specifico del laboratorio. In questo modo la stessa suite può essere eseguita sia in un contesto offline, tramite componenti simulati, sia contro un proxy SIP reale oppure attraverso un percorso di inoltro verso il P-CSCF.

4.1 Scelte di implementazione e motivazioni

La prima scelta significativa è stata adottare Python come linguaggio principale del framework. Questa decisione è stata guidata da tre considerazioni. In primo luogo, Python consente di sviluppare rapidamente componenti di orchestration, parsing e validazione senza introdurre complessità superflua. In secondo luogo, la sua sintassi leggibile facilita la manutenzione del codice e la revisione delle logiche di test. Infine, la disponibilità di librerie mature per la gestione di YAML, HTTP, JSON e test automatici si sposa bene con la natura del progetto, che integra configurazione dichiarativa, trasporto di controllo e produzione di report.

La seconda scelta è stata l'uso di YAML per descrivere sia le suite di test sia il runtime di esecuzione. Questo formato è stato preferito perché esprime in modo chiaro strutture gerarchiche, liste di casi e parametri di configurazione, mantenendo allo stesso tempo un buon compromesso tra leggibilità umana e validabilità automatica. Per una tesi orientata alla sperimentazione, questa caratteristica è fondamentale: la configurazione deve essere comprensibile, modificabile e facilmente confrontabile tra una prova e l'altra.

La terza scelta è stata l'impiego di Docker come base del laboratorio. L'obiettivo non era soltanto isolare le dipendenze, ma rendere replicabile l'intero ambiente di

test. Per questo motivo i componenti del core, i servizi di supporto, i simulatori e il proxy sono stati inseriti in una rete Docker comune, in modo da poter comunicare tra loro in modo deterministico. In particolare, il proxy è stato incluso nella stessa rete dei servizi IMS per poter osservare e instradare correttamente il traffico SIP senza introdurre ulteriori livelli di astrazione. Questa soluzione semplifica il deployment, riduce i problemi legati alla configurazione degli indirizzi e rende più chiaro il percorso dei messaggi durante le prove.

La quarta scelta è stata mantenere il traffico SIP il più possibile vicino alla forma originale. Il framework tratta i messaggi come testo raw e applica trasformazioni minime e controllate soltanto dove necessario. Questa impostazione è stata adottata per non introdurre correzioni automatiche che avrebbero potuto alterare il significato dei test, soprattutto nei casi in cui l'oggetto della verifica è proprio il comportamento del dispositivo davanti a messaggi incompleti, non standard o volutamente malformati.

La quinta scelta è stata rendere il sistema estendibile attraverso astrazioni semplici e ben definite. Il motore di test non conosce i dettagli del backend concreto, ma si limita a usare interfacce comuni per l'invio, la ricezione e la discovery. In questo modo è possibile aggiungere nuovi meccanismi di trasporto senza riscrivere la logica di esecuzione o la definizione delle suite.

Dal punto di vista operativo, queste scelte si traducono in una soluzione che mostra con chiarezza cosa è stato realizzato: un motore di test in Python, una configurazione in YAML, un laboratorio containerizzato in Docker e un proxy integrato nella stessa rete dei servizi IMS. La combinazione di questi elementi consente di passare da una descrizione astratta del sistema a un'implementazione concreta, ripetibile e adatta alla validazione sperimentale.

4.2 Stack tecnologico e organizzazione del codice

L'applicazione di test è distribuita come pacchetto Python e viene esposta tramite un'interfaccia a riga di comando. Il cuore dell'implementazione è organizzato in componenti con responsabilità ben distinte.

Una parte si occupa dell'interazione con l'utente e della gestione delle modalità operative principali: validazione dei test, elencazione dei dispositivi disponibili, esecuzione standard, confronto differenziale e gestione delle regole di live edit. Un'altra parte si occupa del caricamento e della validazione delle configurazioni, mentre il livello di dominio rappresenta suite di test, dispositivi, risposte, report ed errori di configurazione.

La modifica dei messaggi SIP è affidata a un componente dedicato che applica trasformazioni minime e controllate, mentre il motore di esecuzione gestisce sia la verifica di conformità sia il confronto tra due terminali. I trasporti sono raccolti e registrati in modo da rendere aggiungibili nuovi backend senza modificare il flusso principale.

La struttura è volutamente interface-first. Sono definite astrazioni per l'invio e la lettura dei messaggi SIP, per la modifica dei template e per la creazione dei trasporti. Questa scelta riduce il legame tra il motore di test e la rete sottostante e rende possibile sostituire un backend con un altro senza toccare la logica di validazione.

4.3 Configurazione e modello dei dati

Il comportamento del programma è guidato da due famiglie di file YAML. I file di test descrivono il contenuto della suite, ossia i casi da eseguire, i messaggi SIP da inviare, le modifiche da applicare e le risposte attese. I file runtime, invece, descrivono il contesto

di esecuzione: trasporto scelto, parametri di discovery e anagrafica dei dispositivi.

Questa separazione è importante perché consente di riutilizzare la stessa suite su più piattaforme. Un test che verifica, ad esempio, la corretta risposta a una registrazione può essere eseguito sia contro un telefono fisico sia contro un dispositivo scoperto dinamicamente tramite traffico SIP osservato dal proxy. Il runtime decide solo come raggiungerlo e con quale backend parlare.

Durante il caricamento viene applicata una validazione strutturale e semantica. Vengono verificati i campi obbligatori, la forma dei documenti YAML e la coerenza tra configurazione e modalità di esecuzione. In caso di errore vengono restituiti messaggi leggibili, così da semplificare la diagnosi dei problemi di setup.

Il modello dati rappresenta esplicitamente i concetti usati dal dominio: dispositivo, risposta SIP, caso di test, report di compliance e report di confronto. Questa modellazione rende più semplice serializzare i risultati in formato JSON, stampare report leggibili in console e alimentare eventuali strumenti esterni di analisi.

4.4 CLI e flussi operativi

L'applicazione espone un insieme di sottocomandi che riflettono i casi d'uso reali del framework. Sono previste funzioni per controllare la correttezza di una suite YAML, mostrare i dispositivi configurati e quelli eventualmente scoperti, eseguire una suite su uno o più dispositivi e lanciare il confronto differenziale tra due terminali.

Sono presenti anche funzioni dedicate alle regole di live edit, che consentono di ispezionare e ripulire le regole di intercettazione mantenute dal proxy SIP. Questa funzionalità è utile soprattutto durante il debugging, perché permette di verificare lo stato del proxy senza accedere direttamente ai suoi meccanismi interni.

Il flusso di esecuzione in modalità standard inizia caricando suite e runtime. Suc-

cessivamente il programma attiva il trasporto richiesto, apre la connessione e prepara il modificatore SIP di default. Se il runtime supporta la discovery dei dispositivi, la lista disponibile viene aggiornata prima della selezione. Quando l'utente non specifica un singolo device, l'interfaccia permette di scegliere uno o più dispositivi dalla lista e, in presenza di selezione multipla, passa automaticamente alla comparazione differenziale.

In questo modo il comportamento della CLI rimane coerente con il dominio: una suite può essere eseguita come test di conformità puntuale oppure come strumento di confronto tra due terminali, senza duplicare il codice di invio e validazione.

4.5 Motore di esecuzione dei test

Il motore di esecuzione è responsabile della traduzione tra configurazione e azioni concrete sul trasporto. Una modalità esegue i casi previsti dalla suite su un singolo dispositivo, mentre l'altra ripete gli stessi input su due dispositivi e confronta gli esiti ottenuti. Entrambe utilizzano la stessa interfaccia di trasporto e lo stesso modificatore dei messaggi, così da mantenere allineata la logica di costruzione dei test vettoriali.

Il ciclo di esecuzione segue una sequenza precisa: preparazione del messaggio SIP, applicazione delle modifiche definite nel test, invio del messaggio tramite backend, attesa delle risposte e confronto con il risultato atteso. Questa sequenza è stata pensata per distinguere chiaramente tre livelli: il contenuto del test, il trasporto di rete e la logica di validazione.

Nei casi in cui il test richiede attesa interattiva tra un caso e il successivo, il runner può sospendere l'esecuzione fino alla conferma dell'operatore. Questo comportamento è utile durante campagne manuali o quando si vuole osservare sul campo il traffico prodotto dal dispositivo sotto test.

L'output finale viene convertito in report di compliance o di comparazione e, se richiesto, può essere scritto anche in formato JSON. In questo modo i risultati non restano confinati alla sola console ma possono essere consumati da pipeline esterne o archiviati per successive analisi.

4.6 Gestione dei messaggi SIP

Uno degli aspetti più delicati dell'implementazione riguarda la manipolazione dei messaggi SIP, in particolare quando i casi di test devono preservare comportamenti non perfettamente conformi o messaggi deliberatamente malformati. Il sistema opera sul template testuale del messaggio senza ricostruire completamente il contenuto a partire da una struttura astratta, proprio per evitare di normalizzare campi che il test vuole mantenere invariati.

L'approccio seguito è quindi conservativo: il messaggio viene trattato come testo SIP raw e le modifiche vengono applicate solo dove esplicitamente richiesto dalla suite. Questo consente di testare anche scenari di robustezza, ad esempio intestazioni duplicate, valori incoerenti o formati parzialmente invalidi, senza che il sistema corregga automaticamente il payload.

Il rendering supporta anche placeholder dinamici legati al dispositivo selezionato. In questo modo un singolo template può essere riutilizzato su più terminali e adattarsi all'identità del device in esecuzione. La sostituzione dei placeholder viene eseguita in fase di preparazione del caso, prima dell'invio al backend, così da mantenere il comportamento deterministico e tracciabile.

4.7 Backend di trasporto e protocolli di comunicazione

L'implementazione supporta più backend, ognuno pensato per un contesto operativo diverso. Un backend simulato è utile in fase di sviluppo e test automatici: non contatta elementi di rete reali e restituisce risposte SIP preconfigurate. Questo rende possibile esercitare la soluzione anche in assenza del laboratorio completo.

Un altro backend funge da ponte tra il programma di test e un servizio di proxy che espone una API HTTP dedicata. In questo caso il messaggio SIP viaggia come stringa raw dentro un payload JSON, mentre la comunicazione di controllo usa HTTP/1.1. Il client verifica lo stato del proxy, invia i messaggi, legge le risposte e gestisce l'inventario dei dispositivi tramite endpoint dedicati. Le regole di live edit sono gestite con operazioni specifiche, inclusa l'attesa di eventi di intercettazione.

Un terzo backend rappresenta una modalità di inoltro verso il P-CSCF. In questo scenario il programma di test inserisce un'indicazione dedicata per specificare la destinazione del messaggio e lascia al proxy il compito di inoltrarlo verso il terminale selezionato. Questa soluzione consente di mantenere il comportamento IMS del proxy inalterato per il traffico normale, aggiungendo al tempo stesso un punto di iniezione controllato per i test.

Dal punto di vista dei protocolli, il progetto utilizza quindi un insieme eterogeneo di tecnologie coerenti con l'ambiente IMS e 5G: SIP/2.0 su UDP o TCP per i messaggi applicativi, HTTP/1.1 e JSON per il canale di controllo con il proxy, Diameter su TCP o SCTP per il collegamento con gli elementi core che gestiscono identità e autenticazione, e i protocolli di rete mobile tipici della piattaforma Open5GS, come SBI, NGAP, PFCP e GTP-U, per l'interazione tra i nodi di core network.

4.8 Piattaforma Docker e deployment

La base di esecuzione del progetto è Docker, scelto per rendere replicabile l'intero laboratorio e per isolare le dipendenze dei singoli componenti. I servizi sono descritti in più manifest di composizione, uno per ciascuno scenario: sola rete 5G, rete 4G con VoLTE, VoNR, integrazioni con OpenSIPS, e varianti con ePDG, OCS o componenti di simulazione radio.

La composizione dei servizi segue una logica modulare: i container del core network, i database, i componenti IMS e i simulatori di accesso radio vengono avviati come unità indipendenti ma collegate da reti Docker dedicate. Le configurazioni dei singoli nodi vengono generate a partire da template contenenti parametri segnaposto, sostituiti all'avvio da script di inizializzazione tramite variabili d'ambiente. Questo approccio semplifica il riuso dei medesimi file tra ambienti differenti, cambiando solo indirizzi IP, MCC, MNC, port e range di rete.

Sul piano operativo, sono previste immagini base e manifest di deployment per gli scenari più rilevanti. Il laboratorio può essere avviato su un singolo host oppure distribuito su più macchine, ad esempio separando il nodo radio dall'insieme EPC/5GC e IMS. Sono previste configurazioni sia OTA sia RF-simulated, con supporto a srsRAN, UERANSIM e ad altri elementi di test.

Questa impostazione rende la piattaforma sufficientemente flessibile per il lavoro di validazione: lo stesso framework di test può essere eseguito in un ambiente leggero, basato su backend simulati, oppure in un deployment completo che coinvolge l'intera catena di segnalazione.

4.9 Validazione automatica e manutenzione

Per ridurre il rischio di regressioni, il progetto include una suite di test automatici. I test coprono il caricamento della configurazione, il parsing dei file YAML, la logica dell'interfaccia a riga di comando, la modifica dei messaggi SIP, i backend di trasporto e la generazione dei report. Questa copertura non serve solo a verificare il corretto funzionamento delle singole parti, ma anche a documentare il comportamento atteso dei componenti principali.

La presenza di esempi pronti all'uso è stata fondamentale anche dal punto di vista manutentivo. Ogni nuovo scenario può essere descritto attraverso una combinazione di suite YAML e runtime YAML, senza introdurre nuove parti di logica nel motore di esecuzione. In altre parole, l'estensione del framework passa più spesso dalla configurazione che dalla modifica dell'applicazione.

4.10 Sintesi

L'implementazione del framework è stata costruita per coniugare flessibilità e robustezza. Python fornisce il livello applicativo del sistema di test, YAML descrive in modo dichiarativo test e runtime, gli script di inizializzazione e Docker si occupano del deployment della piattaforma, mentre SIP, HTTP, Diameter e gli altri protocolli di rete caratterizzano l'integrazione con il laboratorio IMS e 5G.

La scelta più importante è stata quella di separare chiaramente la logica di test dal trasporto sottostante. Grazie a questa impostazione il sistema può operare sia in modalità offline, sia contro un proxy SIP reale, sia in uno scenario di forwarding verso il P-CSCF. Questa modularità costituisce la base tecnica che rende il framework estendibile e adatto a campagne di validazione successive.

Capitolo 5

Validazione e risultati

Questo capitolo riporta i test eseguiti sul framework e i risultati ottenuti in termini di correttezza, robustezza e comportamento osservato.

Descrivere l'ambiente di test, i casi di prova principali e l'analisi dei risultati ottenuti durante la validazione.

Capitolo 6

Conclusioni e sviluppi futuri

Questo capitolo riassume i risultati ottenuti, valuta l'efficacia del framework sviluppato e propone possibili estensioni future.

Riassumere i risultati del progetto, discutere i limiti del lavoro svolto e indicare possibili evoluzioni del framework.

Elenco delle figure

Elenco dei codici

Elenco degli algoritmi

Bibliografia

- [1] N. Cognome, “Dummy title,” 05 2003.